

Milestone 5 – Row-Level Authorization and Access Control Design

Overview

The Customer API enforces authorization based on both the authenticated user's role and the ownership or access relationship of a customer record. This introduces row-level security combined with role-based access control.

Authorization Rules

Role	Access Permissions
Admin	Full access to all customer records
QA	Access to QA-owned records and their own records
Customer	Access restricted to their own customer records only

Planned Enhancement: Many-to-Many Access Model

To support scenarios where multiple users must access the same customer record, the system evolves from a single-owner model to a many-to-many access control design.

Step 1 – Create CustomerAccess Table

A dedicated mapping table is introduced to manage user-to-customer relationships.

```
CREATE TABLE CustomerAccess
(
    CustomerAccessId INT IDENTITY PRIMARY KEY,
    CRMCustomerID NVARCHAR(50) NOT NULL,
    Username NVARCHAR(100) NOT NULL,
```

```
CONSTRAINT FK_CustomerAccess_Customer
  FOREIGN KEY (CRMCustomerID)
  REFERENCES Customer(CRMCustomerID),

CONSTRAINT FK_CustomerAccess_User
  FOREIGN KEY (Username)
  REFERENCES Users(Username)
);
```

Step 2 – Remove OwnerUsername Dependency

The **OwnerUsername** column is removed as ownership is now managed through **CustomerAccess**.

Before:

Customer
CRM100 | OwnerUsername = michael

After:

Customer
CRM100

CustomerAccess
CRM100 | michael
CRM100 | schan1
CRM100 | qauser

This enables multiple authorized users per customer.

Step 3 – Authorization Logic Flow

Authorization is centralized through a validation flow:

Admin → Allow access

Non-Admin users:

Check whether (CRMCustomerID, CurrentUser) exists in CustomerAccess.

If found → Allow
If not found → Deny

Step 4 – Customer Creation Behavior

When a customer is created:

1. Insert record into Customer table.
2. Automatically insert a corresponding record into CustomerAccess for the creator.

Example:

Customer
CRM100

CustomerAccess
CRM100 | michael

Step 5 – Access Management Endpoints

The following endpoints are introduced to manage customer access:

- POST /api/customer/{crmId}/grant-access
- DELETE /api/customer/{crmId}/revoke-access
- GET /api/customer/{crmId}/users

Only Admin users are permitted to manage access permissions.

Benefits

This enhancement enables:

- Multiple users assigned to a single customer
- Team-based collaboration
- Shared ownership model
- Department-level access control
- Improved scalability and maintainability of authorization logic

Enterprise Architecture Flow

Users

- JWT Authentication
 - Role Claims
 - CustomerAccess Table
 - Row-Level Authorization
 - Customer API
-

Status

This feature is planned for a future milestone.

Milestone 5 focuses on implementing:

- JWT authentication
- BCrypt password hashing
- Database-backed users
- Role-based authorization
- Row-level security using ownership or access mapping

The many-to-many access model is identified as a future enhancement toward a production-grade authorization system.

Alternative Ownership-Based Implementation (Earlier Design)

Step 1 – Add Ownership Column

```
ALTER TABLE Customer  
ADD OwnerUsername NVARCHAR(100) NULL;
```

Step 2 – Authorization Helper

Centralized authorization logic:

```

private bool CanAccessCustomer(string ownerUsername)
{
    var currentUser = User.Identity?.Name;
    var role = User.FindFirst(ClaimTypes.Role)?.Value;

    if (role == "Admin")
        return true;

    if (role == "QA")
    {
        using var conn = new SqlConnection(_connectionString);

        const string sql = "SELECT Role FROM Users WHERE Username=@u";

        using var cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@u", ownerUsername);

        conn.Open();

        var ownerRole = cmd.ExecuteScalar()?.ToString();

        return ownerRole == "QA" || ownerUsername == currentUser;
    }

    return ownerUsername == currentUser;
}

```

This ensures consistent enforcement of authorization rules across the API.

Step 3 – Secure Get Customer Endpoint

Retrieve ownership data and validate access before returning the response.

SQL:

```

SELECT CRMCustomerID,
       FirstName,
       LastName,
       Email,
       Phone,
       OwnerUsername
FROM Customer

```

```
WHERE CRMCustomerID=@CRMCustomerID  
AND IsDeleted=0;
```

Validation:

```
var owner = reader["OwnerUsername"].ToString();
```

```
if (!CanAccessCustomer(owner))  
    return Forbid();
```

Step 4 – Assign Ownership on Creation

Automatically assign ownership during customer creation:

```
var currentUser = User.Identity?.Name;  
cmd.Parameters.AddWithValue("@OwnerUsername", currentUser);
```

Step 5 – Protect Updates

Before updating, validate ownership:

- Retrieve OwnerUsername from database
 - Validate access using `CanAccessCustomer()`
 - Deny access if unauthorized
-

Step 6 – Protect Deletion

Authorization rules:

- Admin: Full access
 - QA: QA-owned and own records
 - Customer: Own records only
-

Step 7 – Simplified Authorization Model

Replace:

```
[Authorize(Policy = "CustomerOwner")]
```

With:

```
[Authorize]
```

Authorization is now fully enforced through application logic.

Final Security Model

Role	Permissions
Admin	Full access to all customer records
QA	Access to QA-owned and own records
Customer	Access restricted to own records

Enterprise Security Features Demonstrated

- JWT Bearer Authentication
 - BCrypt Password Hashing
 - Database-backed Identity
 - Role Claims
 - Row-Level Authorization
 - Multi-Tenant Access Control
 - Stored Procedures
 - SQL Server Integration
 - Soft Delete Pattern
 - Audit Logging
-

Interview Summary

This implementation demonstrates row-level authorization using JWT role claims combined with resource ownership and access mapping. It ensures administrators have unrestricted access while enforcing strict, data-driven access control for QA and Customer roles.

Implementation Status Summary

- Row-level security fully enforced
- Admin bypass implemented
- QA and Customer restrictions applied
- CustomerAccess table integrated
- Ownership logic standardized
- Authorization applied consistently across endpoints
- Automatic access assignment on creation implemented